

Lecture 2:

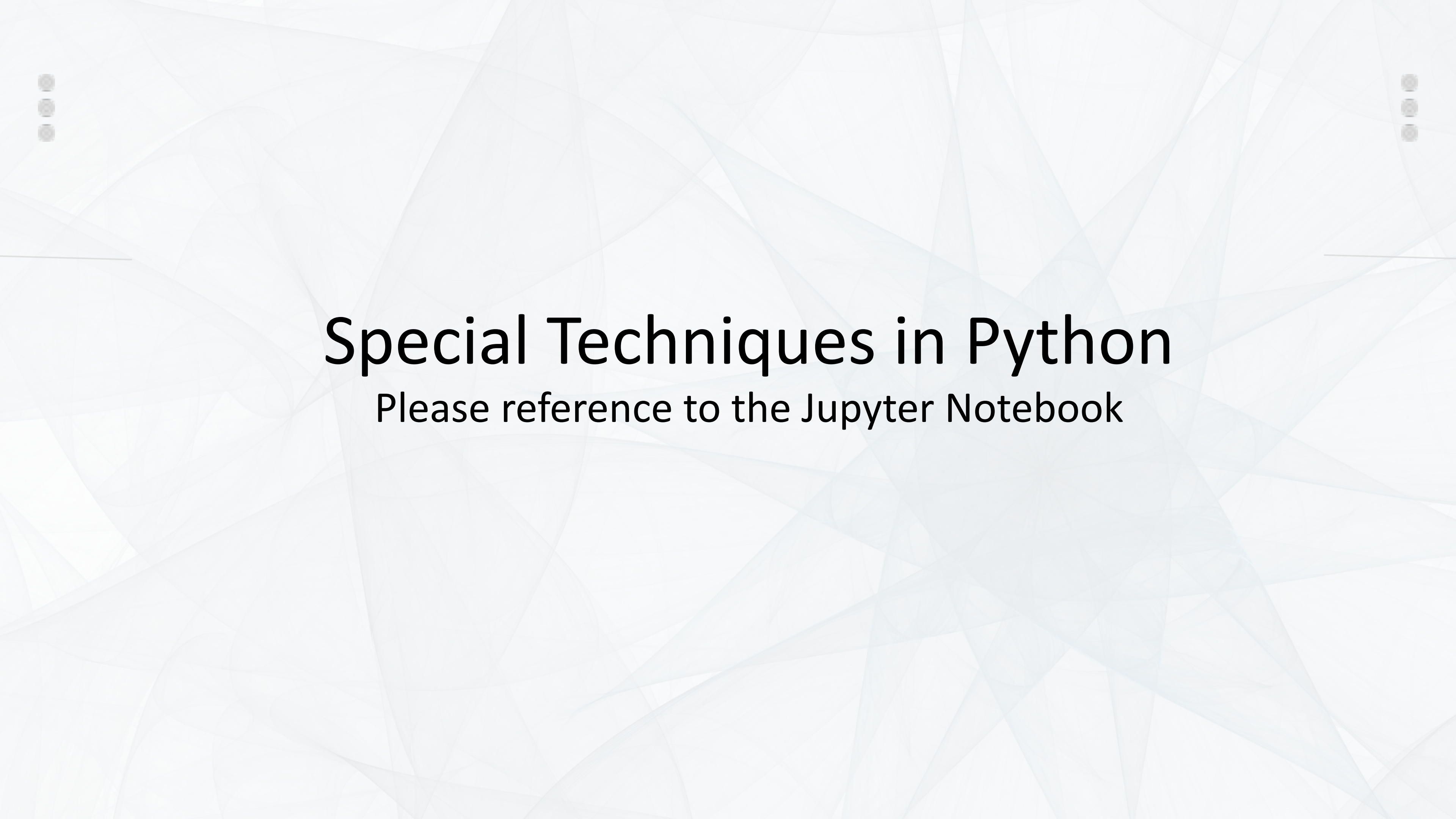
Selected Topics in Python And Programming

Asla Quant
Academy



Agenda

- *Special Techniques in Python*
 - *Magic Methods*
 - *Context Manager*
 - *Generator*
 - *Lambda*
 - *Decorator*
- *Types of Programming Languages*
- *Python Internal*
 - *Overview: From Source Code to Execution*
 - *GIL, Performance, Memory Management*



Special Techniques in Python

Please reference to the Jupyter Notebook

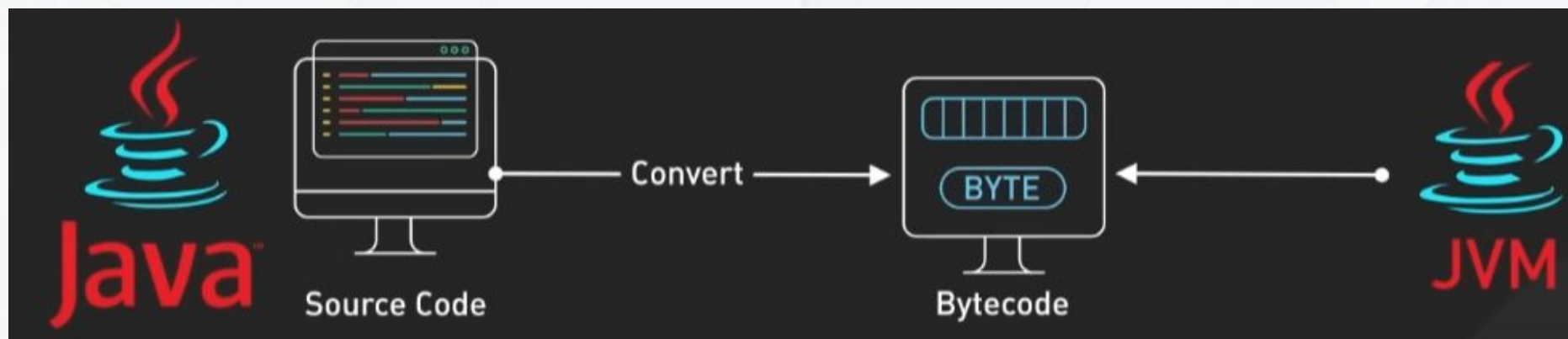
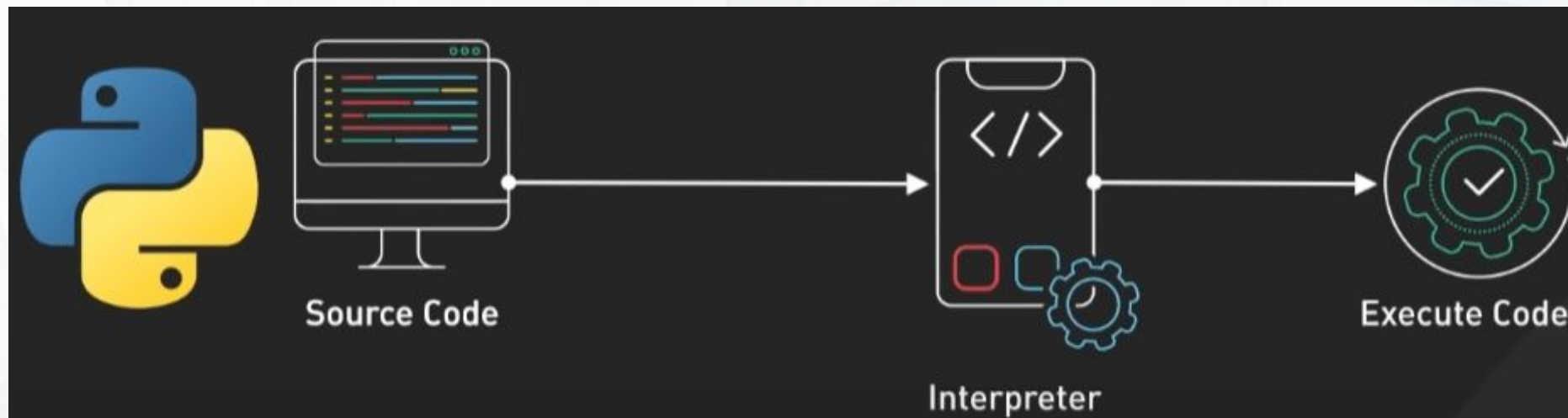
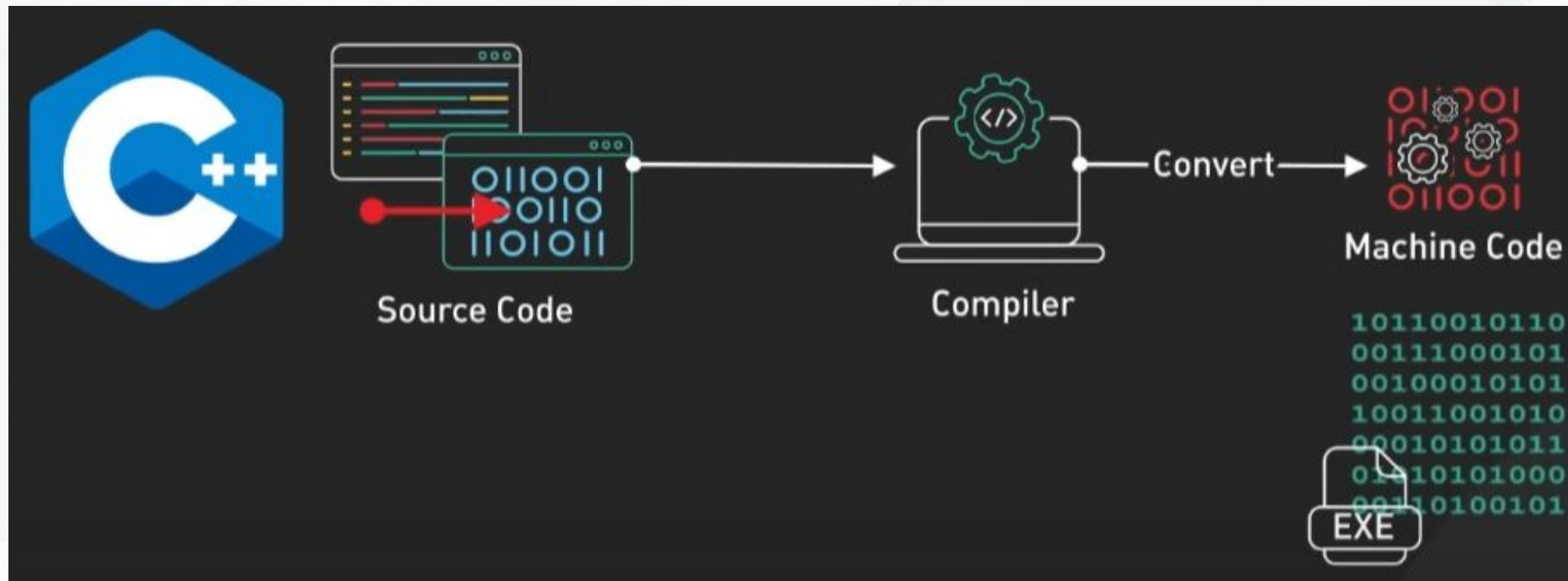


Types of Programming Languages

Compiled & Interpreted Languages

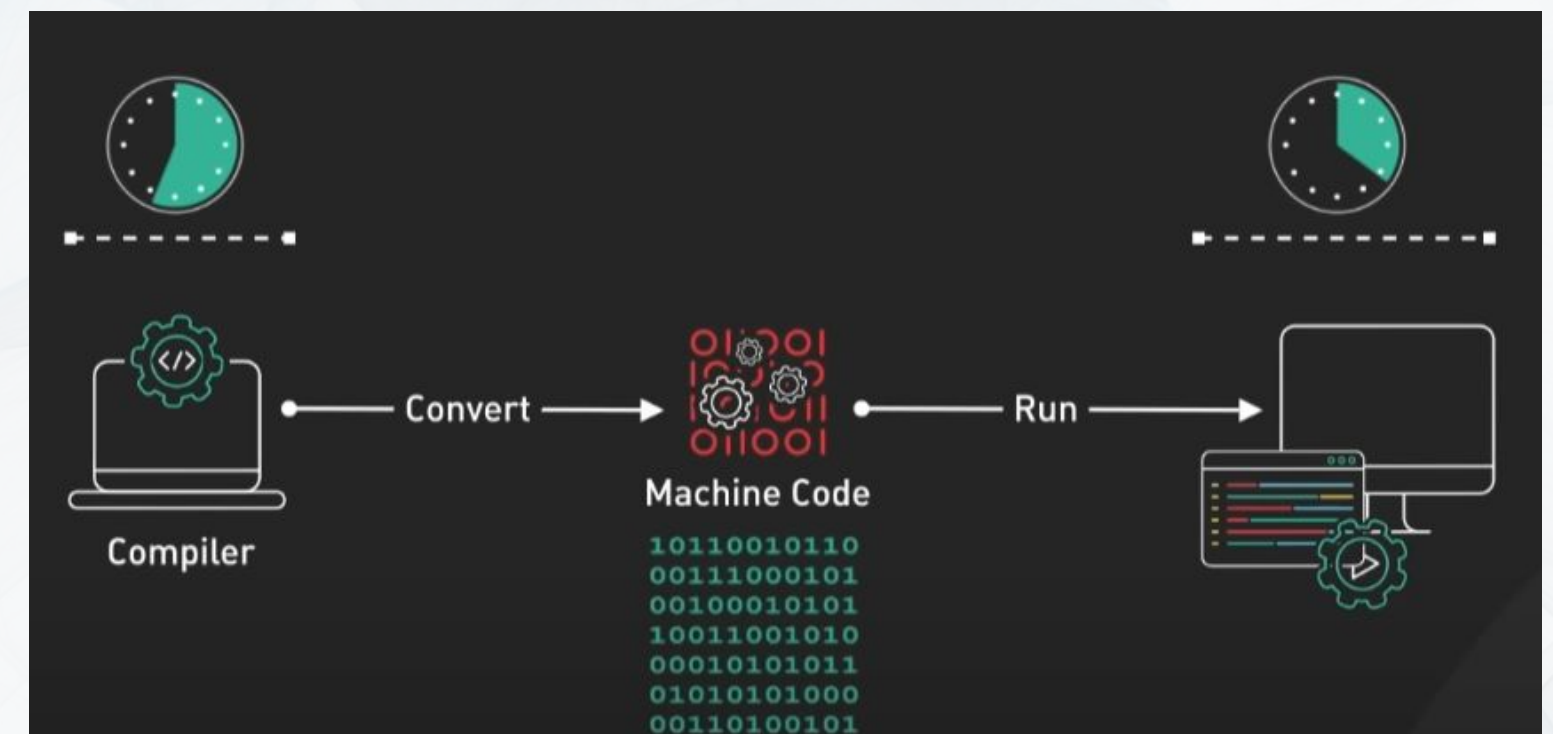
Programming Languages

Python vs C++ vs Java



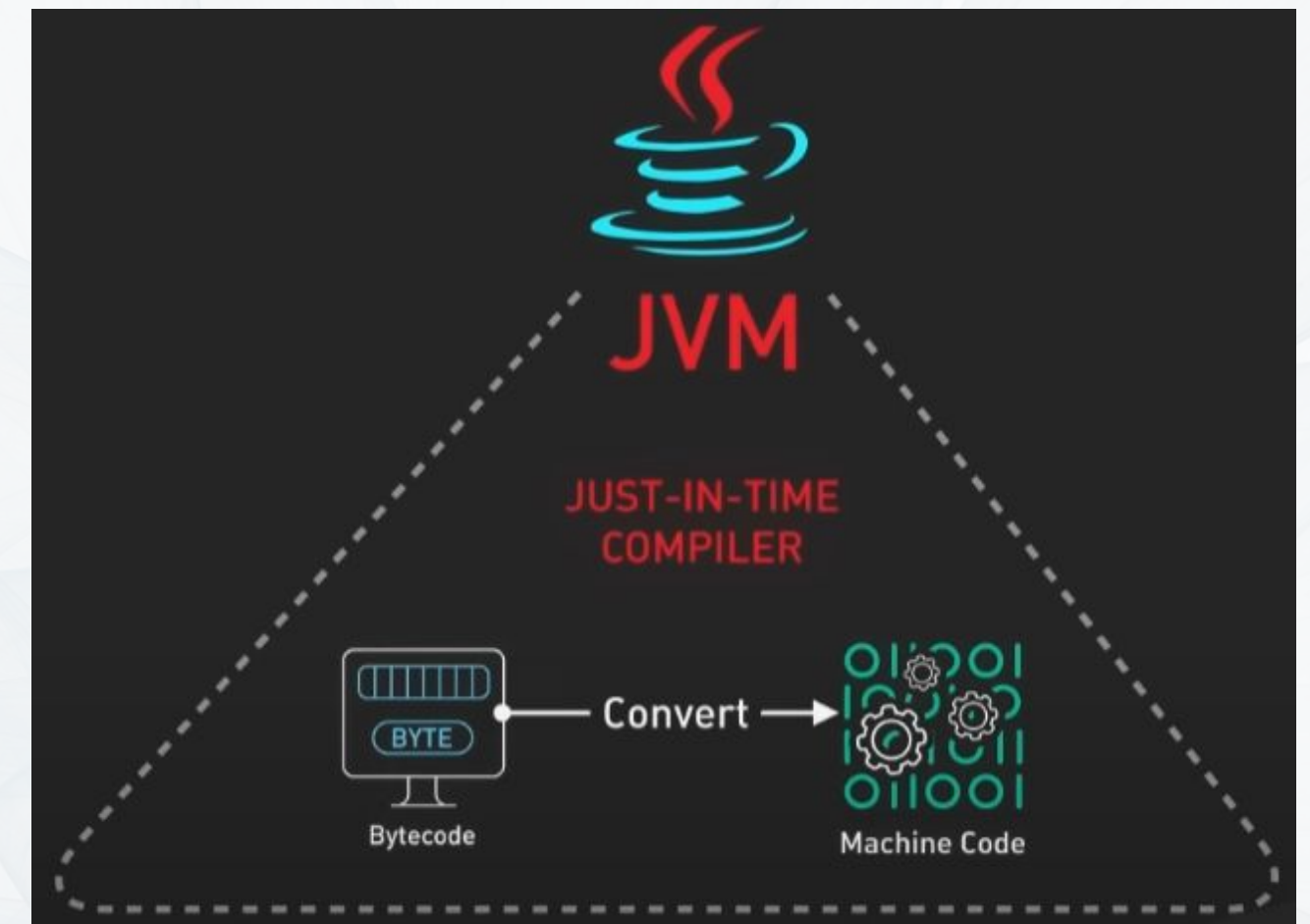
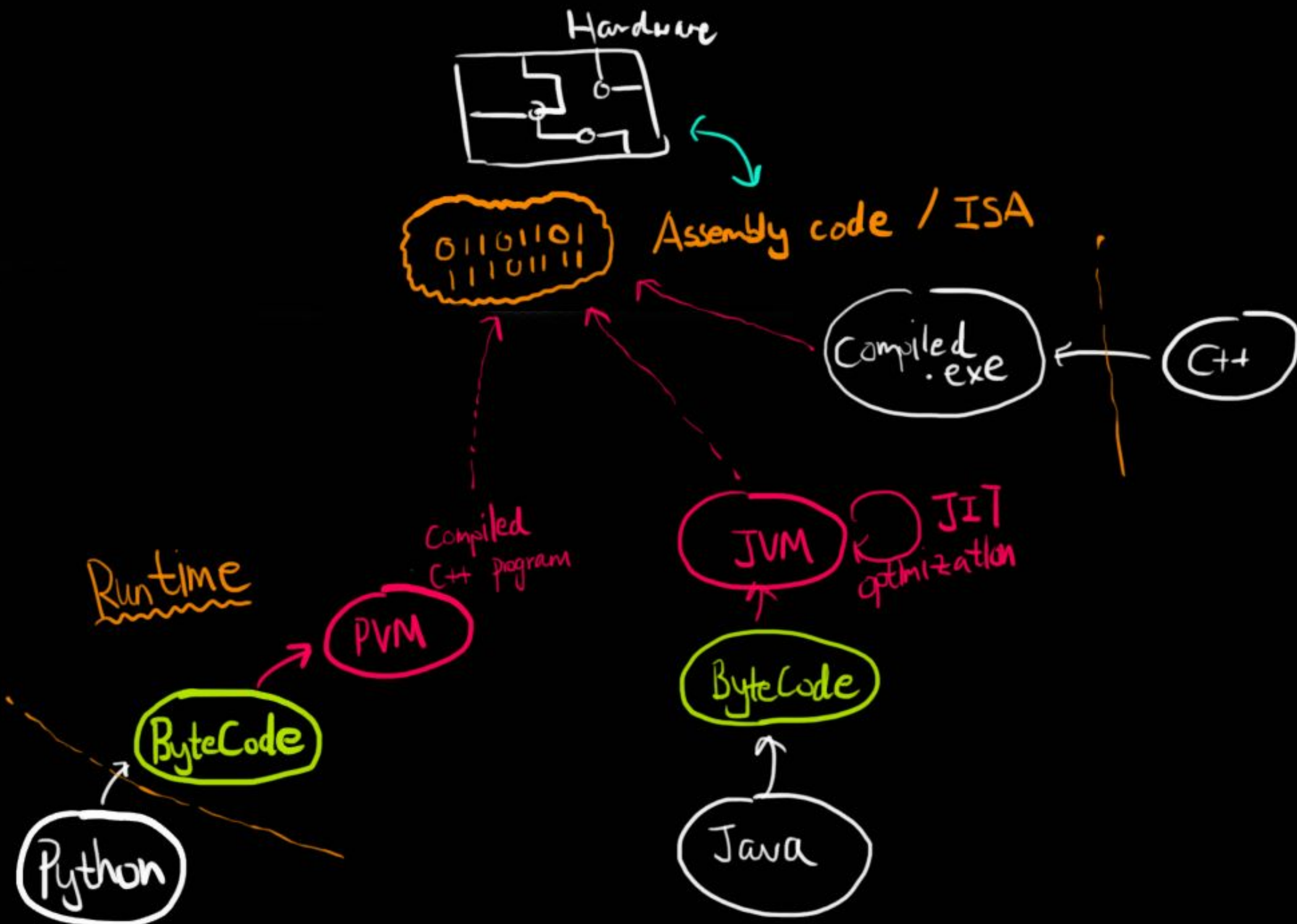
Compiled language - a programming language in which one writes the code, and then “compiles” it into a separate file (program) that the computer can understand, which then gets executed.

Interpreted language - a programming language in which one writes the code, and that code gets “interpreted” into what the computer can understand at runtime. No separate file is created that one has to execute, like with a compiled language.



Programming Languages

Python vs C++ vs Java





Python Internal

Source Code to Execution, GIL, Performance Memory

Management

Python Internal

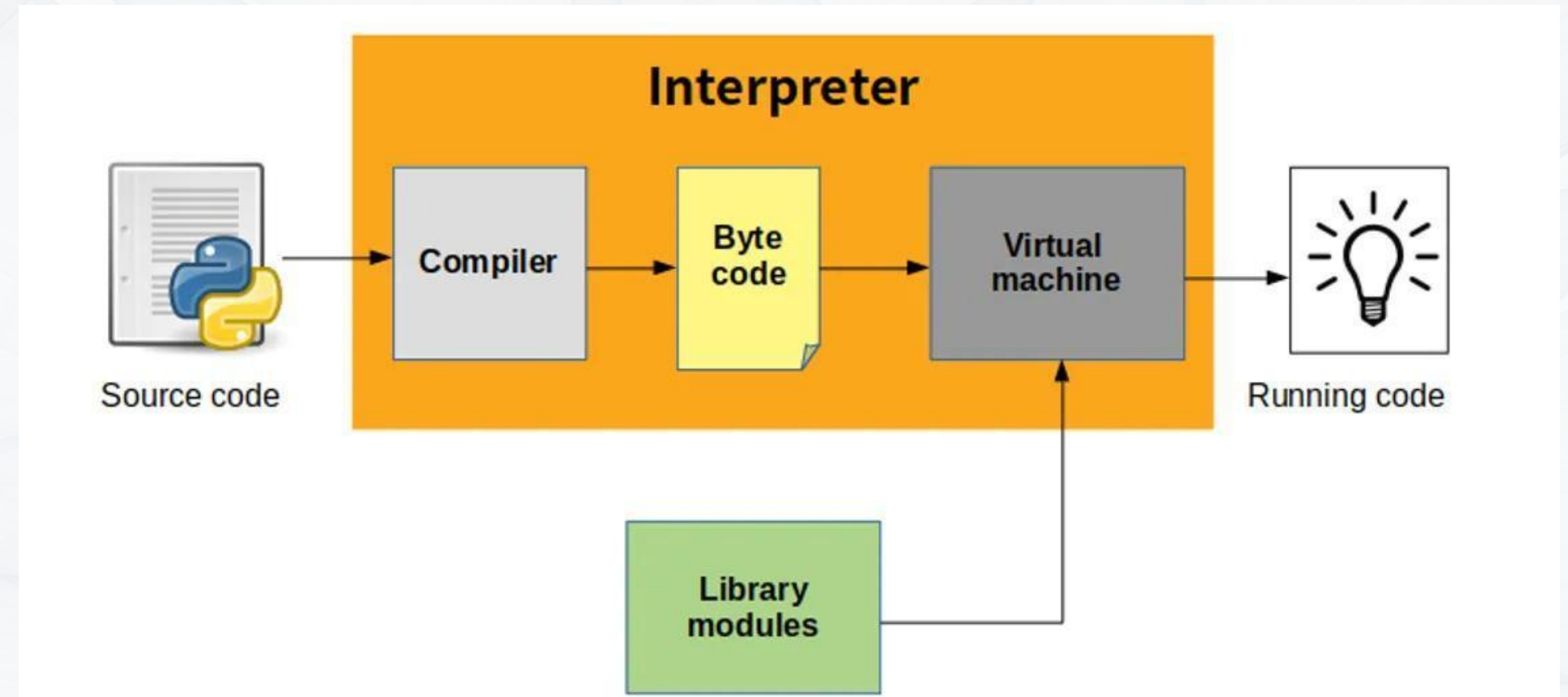
Compiler vs Interpreter

An interpreter is a computer program, which converts each high-level program statement into the machine code. This includes source code, pre-compiled code, and scripts.

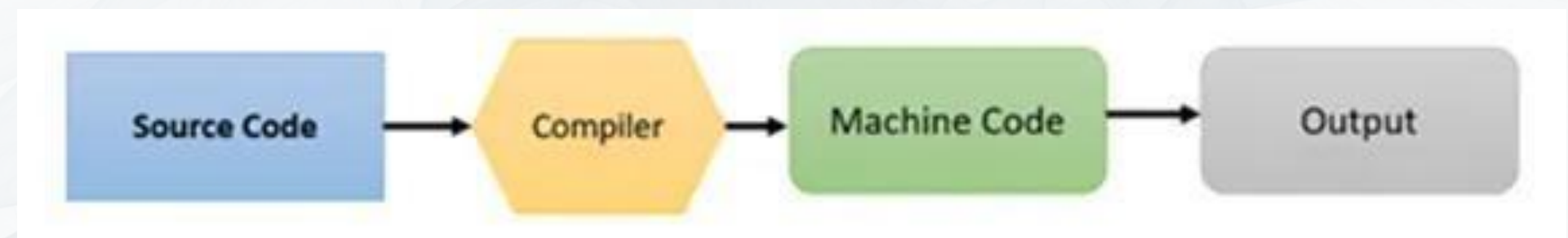
Both compiler and interpreters do the **same** job which is **converting higher level programming language to machine code**.

However, a compiler will convert the code into machine code before program run. Interpreters convert code into machine code when the program is run.

Interpreter:



Compiler:



Python Internal

How Python Works?

```
print("My first Blog on Python Internal workings")
a = 10
b = 20
c = a + b
print("c = ", c)
~
```

Byte code is a virtualized machine code. Unlike machine code for a real processor, byte code is often for an idealized or virtual processor that doesn't actually exist. Byte code is based on a CPU architecture like a register or stack machine but often uses general features common to any CPU or instructions and concepts that don't exist on any CPU.

```
(kaushik@kali) - [~]
$ python3 -m dis bytecode.py
1      0 LOAD_NAME           0 (print)
      2 LOAD_CONST          0 ('My first Blog on Python Internal workings')
      4 CALL_FUNCTION        1
      6 POP_TOP

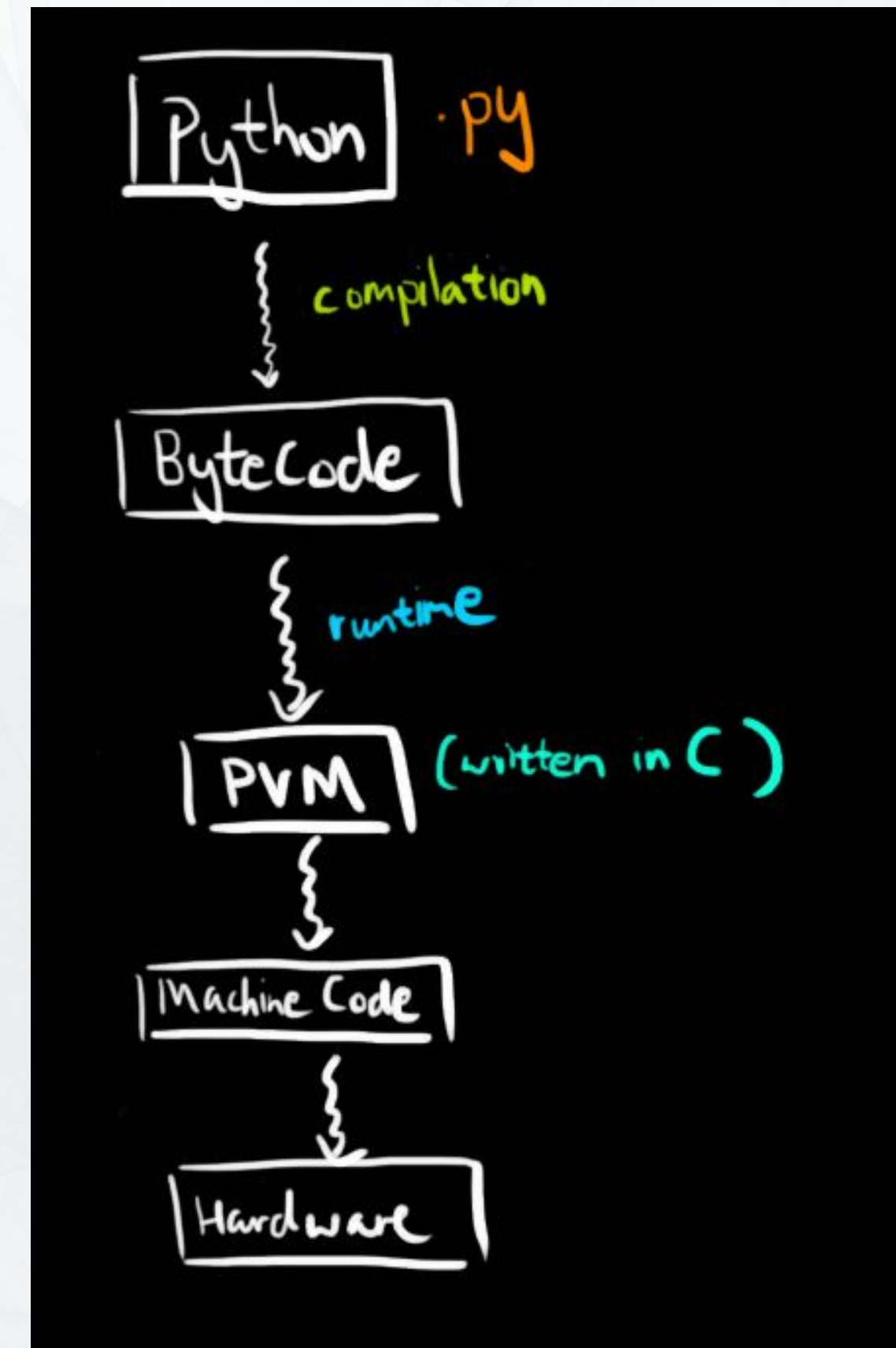
2      8 LOAD_CONST          1 (10)
     10 STORE_NAME         1 (a)

3     12 LOAD_CONST          2 (20)
     14 STORE_NAME         2 (b)

4     16 LOAD_NAME           1 (a)
     18 LOAD_NAME           2 (b)
     20 BINARY_ADD
     22 STORE_NAME         3 (c)

5     24 LOAD_NAME           0 (print)
     26 LOAD_CONST          3 ('c = ')
     28 LOAD_NAME           3 (c)
     30 CALL_FUNCTION        2
     32 POP_TOP
     34 LOAD_CONST          4 (None)
     36 RETURN_VALUE

(kaushik@kali) - [~]
```



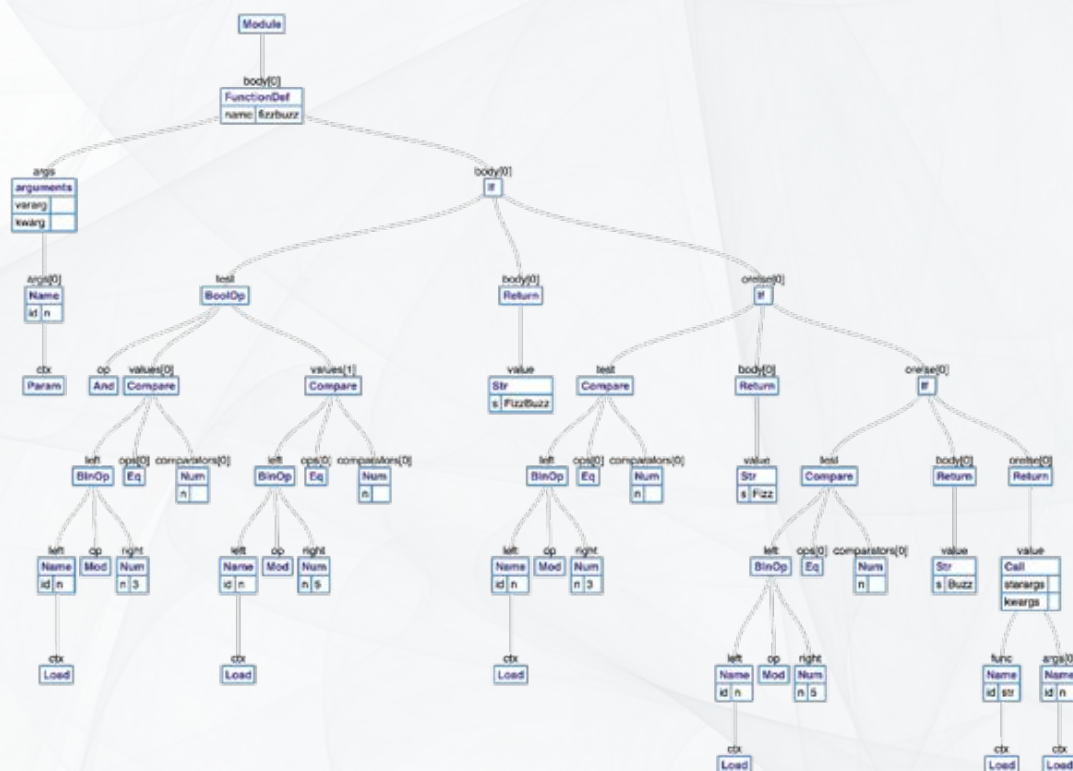
Python Internal

Abstract Syntax Tree to Byte Code

Python Program

```
1 def fizzbuzz(n):
2     if n % 3 == 0 and n % 5 == 0:
3         return 'FizzBuzz'
4     elif n % 3 == 0:
5         return 'Fizz'
6     elif n % 5 == 0:
7         return 'Buzz'
8     else:
9         return str(n)
```

Abstract Syntax Tree (AST)



Compiled Byte Code

```
0 LOAD_FAST 0 (n)
2 LOAD_CONST 1 (3)
4 BINARY_MODULO
6 LOAD_CONST 2 (0)
8 COMPARE_OP 2 (==)
10 POP_JUMP_IF_FALSE 28
12 LOAD_FAST 0 (n)
14 LOAD_CONST 3 (5)
16 BINARY_MODULO
18 LOAD_CONST 2 (0)
20 COMPARE_OP 2 (==)
22 POP_JUMP_IF_FALSE 28
24 LOAD_CONST 4 ('FizzBuzz')
26 RETURN_VALUE

28 LOAD_FAST 0 (n)
30 LOAD_CONST 1 (3)
32 BINARY_MODULO
34 LOAD_CONST 2 (0)
36 COMPARE_OP 2 (==)
38 POP_JUMP_IF_FALSE 44
40 LOAD_CONST 5 ('Fizz')
42 RETURN_VALUE

44 LOAD_FAST 0 (n)
46 LOAD_CONST 3 (5)
48 BINARY_MODULO
50 LOAD_CONST 2 (0)
52 COMPARE_OP 2 (==)
54 POP_JUMP_IF_FALSE 60
56 LOAD_CONST 6 ('Buzz')
58 RETURN_VALUE

60 LOAD_GLOBAL 0 (str)
62 LOAD_FAST 0 (n)
64 CALL_FUNCTION 1
66 RETURN_VALUE
68 LOAD_CONST 0 (None)
70 RETURN_VALUE
```

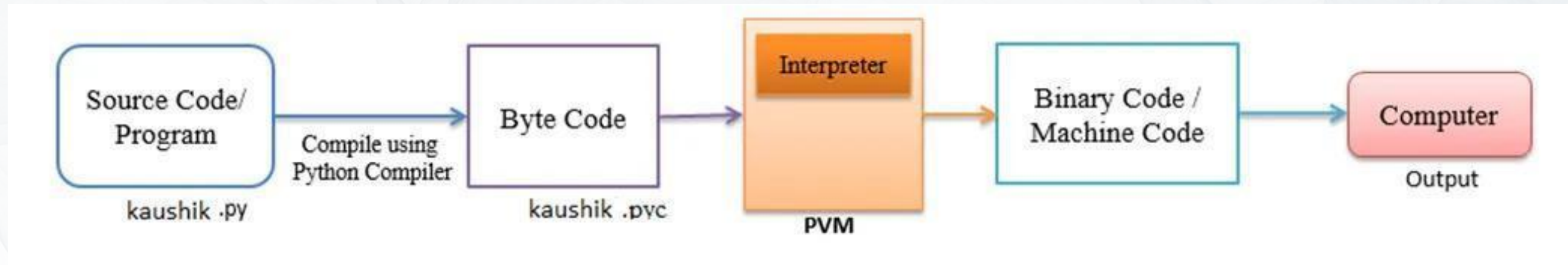
Code Object

```
1 typedef struct basicblock_ {
2     /* Each basicblock in a compilation unit is linked via b_list in the
3     reverse order that the block are allocated. b_list points to the next
4     block, not to be confused with b_next, which is next by control flow. */
5     struct basicblock_ *b_list;
6     /* number of instructions used */
7     int b_iused;
8     /* length of instruction array (b_instr) */
9     int b_ialloc;
10    /* pointer to an array of instructions, initially NULL */
11    struct instr *b_instr;
12    /* If b_next is non-NULL, it is a pointer to the next
13    block reached by normal control flow. */
14    struct basicblock_ *b_next;
15    /* b_seen is used to perform a DFS of basicblocks. */
16    unsigned b_seen : 1;
17    /* b_return is true if a RETURN_VALUE opcode is inserted. */
18    unsigned b_return : 1;
19    /* depth of stack upon entry of block, computed by stackdepth() */
20    int b_startdepth;
21    /* instruction offset for block, computed by assemble_jump_offsets() */
22    int b_offset;
23 } basicblock;
```


Python Internal

Python Virtual Machine (PVM)

Python Virtual Machine (PVM) takes those byte codes converts those instructions into machine code so that the computer can execute those machine code instructions and display the final output.



To carry out this conversion, PVM is equipped with an interpreter.

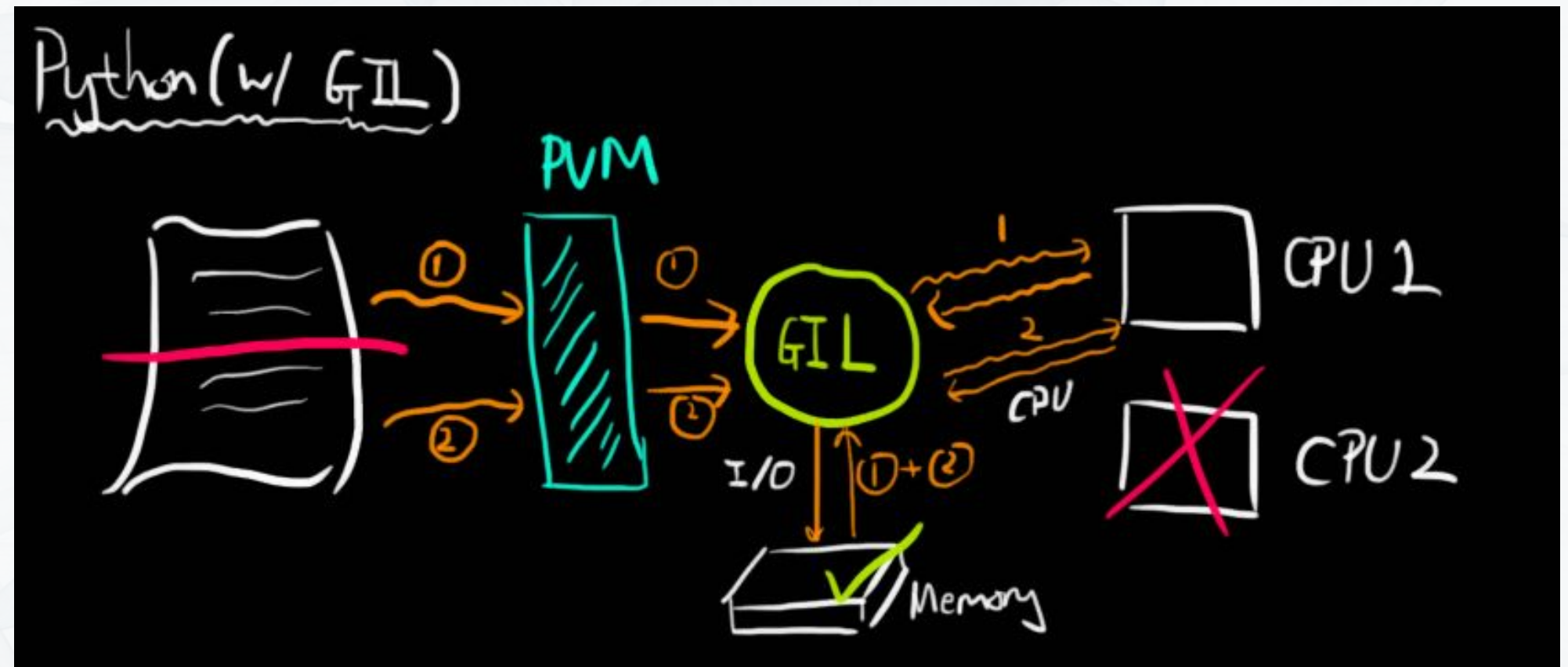
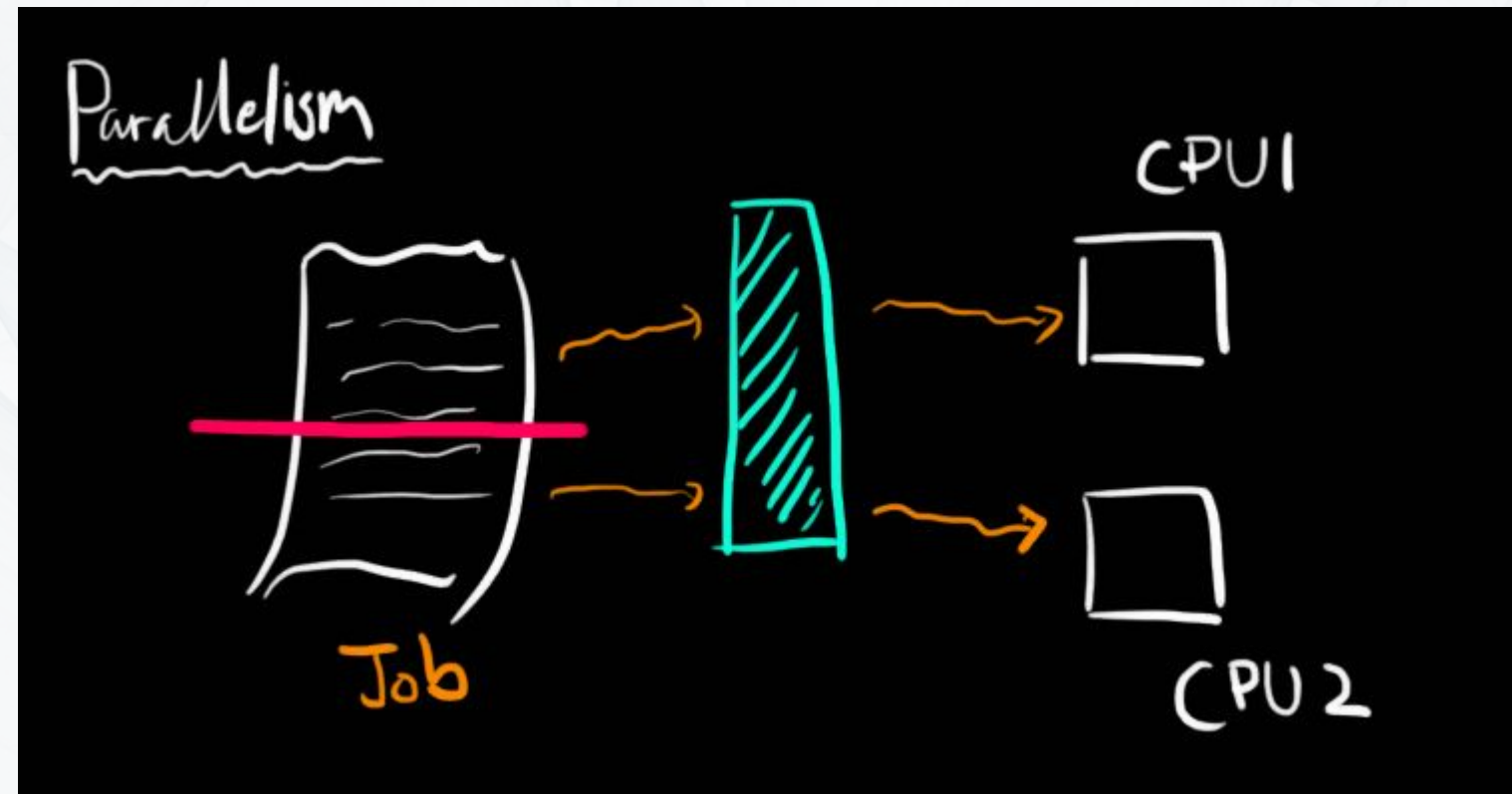
The interpreter converts the byte code into machine code and ,sends that machine code to the computer processor for execution.

Python Internal

CPython & GIL

GIL Behavior

- It's simple : threads hold the GIL when running
- However, they release it when blocking for I/O
- So, any time a thread is forced to wait, other "ready" threads get their chance to run
- Basically, a kind of "cooperative" multitasking

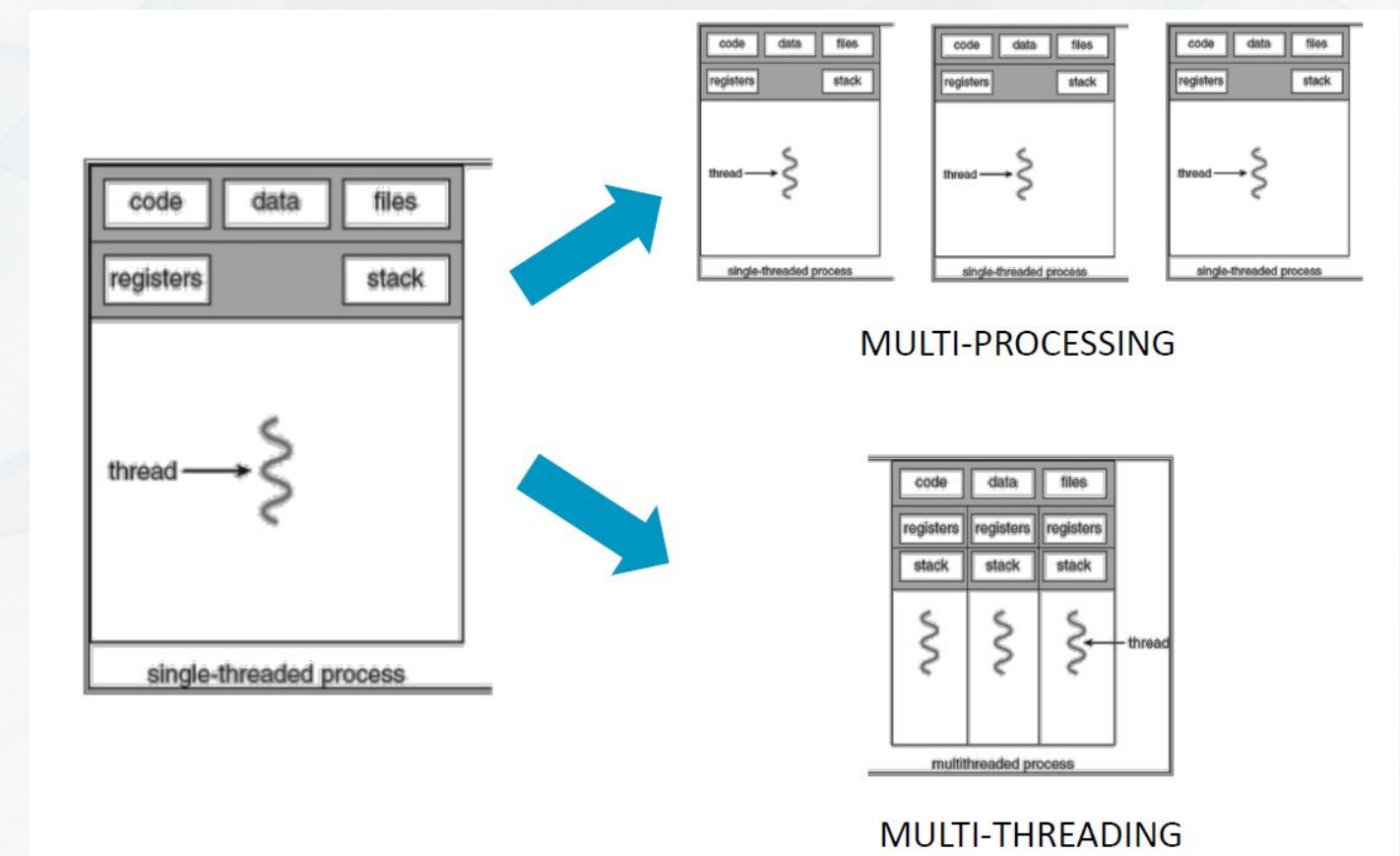
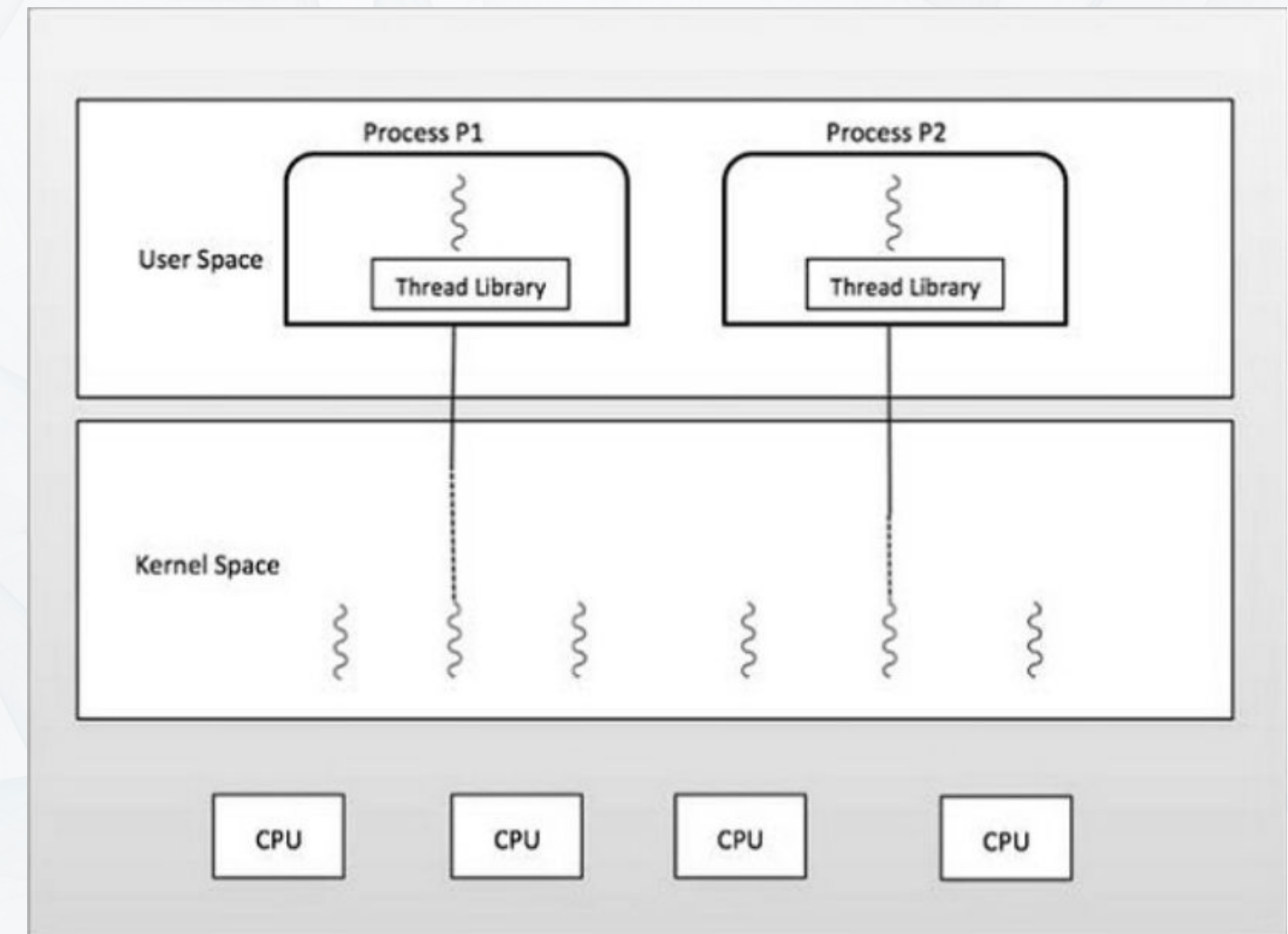


Python Internal

Multiprocessing & Performance

Multi-processing Basics - What is a Process?

- A process is an instance of a computer program that is being executed
- A process can have 1 or several threads (1 in this hands-on)
- The kernel of the OS schedule threads to multiple cores
- The CPython implementation (called CPI, C Python Interpreter) is not thread-safe and only permits a single thread to run at a time
- The standard Python library has two main modules for parallel computing:
 - threading
 - Good for I/O bound tasks
 - Subject to the Global Interpreter Lock (GIL)
 - multiprocessing
 - Circumvents the GIL
 - Useful for parallel computation



Python Internal

Numpy & Interops

Why is Numpy not affected by the GIL?

- The GIL only locks Python objects to ensure thread safety. If there is no need to access Python objects, the GIL can be released.
- Numpy uses a C extension implementation that does not require accessing Python objects. Therefore, in the C code implementation, the GIL can be directly released. It is acquired again when there is a need to access Python objects. The C-API provides macros for releasing and acquiring the GIL.

